1 / 8

# Values and offsets are different.

Follow selected values into their storage positions.

> The strip wraps for display; storage order continues.

## x: chosen row-major storage

Large labels: values · small labels: element offsets

| | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|
| 1.0<br>0 | 2.0<br>1 | 3.0<br>2 | 4.0<br>3 | 5.0<br>4 | 6.0<br>5 | 7.0<br>6 | 8.0<br>7 | 9.0<br>8 | 10.0<br>9 |
| 11.0<br>10 | 12.0<br>11 | 13.0<br>12 | 14.0<br>13 | 15.0<br>14 | 16.0<br>15 | 17.0<br>16 | 18.0<br>17 | 19.0<br>18 | 20.0<br>19 |
| 21.0<br>20 | 22.0<br>21 | 23.0<br>22 | 24.0<br>23 | 25.0<br>24 | | | | | |

Read left to right; wrapped lines continue the same array.

## Value 1.0 is at element offset 0

25 FP32 values × 4 bytes = 100 bytes



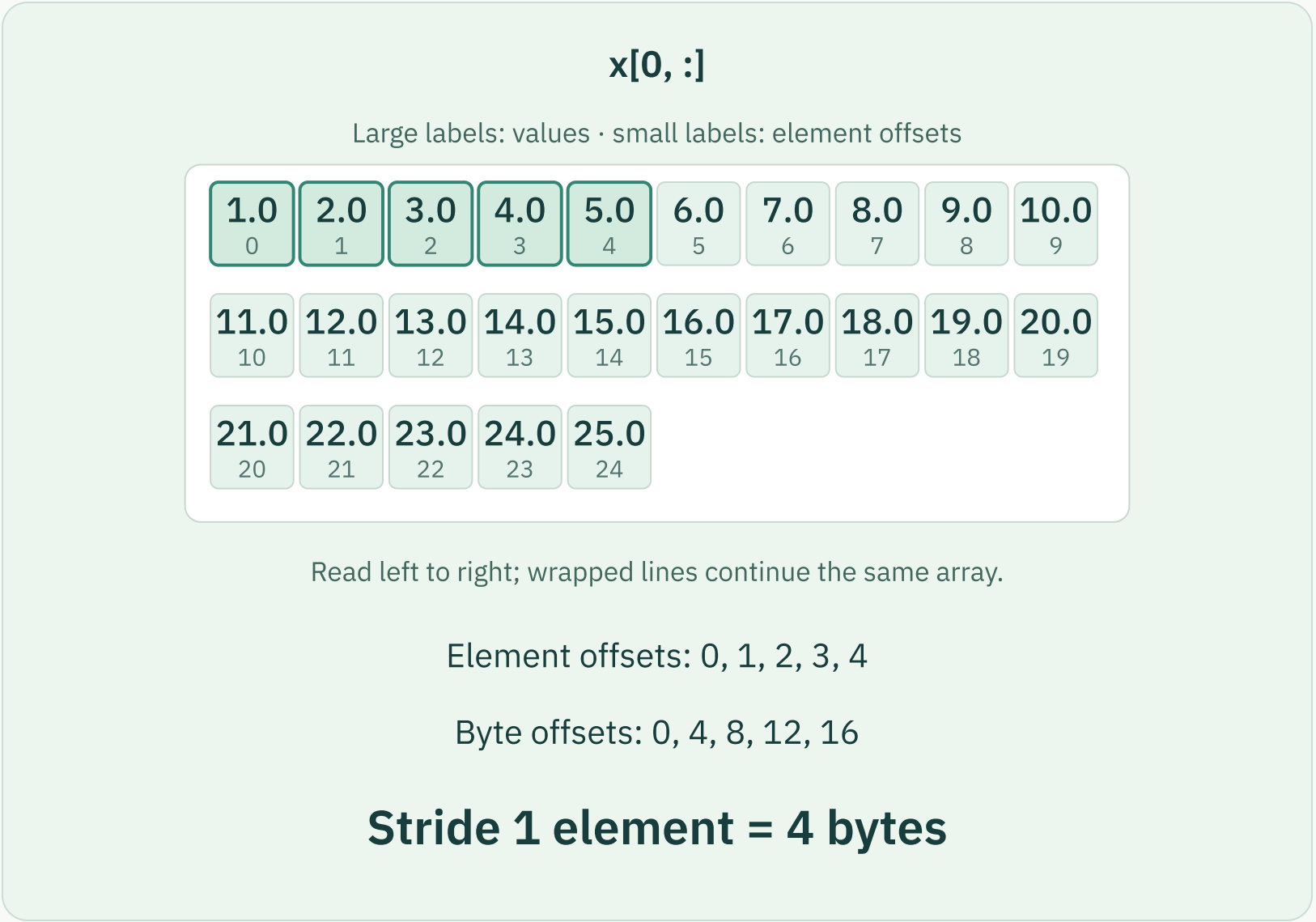
Copyright © 2026 Christopher Snow

2 / 8

# A row uses consecutive positions.

The row view shares the original storage.

> A view uses existing values without moving them.






Copyright © 2026 Christopher Snow

3 / 8

# A column keeps the gaps.

Each next selected value is five element positions away.

> Selecting a column preserves the original positions.

## x[:, 0]

Large labels: values · small labels: element offsets

| | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|
| 1.0<br>0 | 2.0<br>1 | 3.0<br>2 | 4.0<br>3 | 5.0<br>4 | 6.0<br>5 | 7.0<br>6 | 8.0<br>7 | 9.0<br>8 | 10.0<br>9 |
| 11.0<br>10 | 12.0<br>11 | 13.0<br>12 | 14.0<br>13 | 15.0<br>14 | 16.0<br>15 | 17.0<br>16 | 18.0<br>17 | 19.0<br>18 | 20.0<br>19 |
| 21.0<br>20 | 22.0<br>21 | 23.0<br>22 | 24.0<br>23 | 25.0<br>24 | | | | | |

Read left to right; wrapped lines continue the same array.

Element offsets: 0, 5, 10, 15, 20

Byte offsets: 0, 20, 40, 60, 80

**Stride 5 elements = 20 bytes**



Copyright © 2026 Christopher Snow

4 / 8

# Change the start, keep the stride.

Column one starts one element into the same storage.

> The starting offset and the stride are different quantities.

## x[:, 1]

Large labels: values · small labels: element offsets

| | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|
| 1.0<br>0 | 2.0<br>1 | 3.0<br>2 | 4.0<br>3 | 5.0<br>4 | 6.0<br>5 | 7.0<br>6 | 8.0<br>7 | 9.0<br>8 | 10.0<br>9 |
| 11.0<br>10 | 12.0<br>11 | 13.0<br>12 | 14.0<br>13 | 15.0<br>14 | 16.0<br>15 | 17.0<br>16 | 18.0<br>17 | 19.0<br>18 | 20.0<br>19 |
| 21.0<br>20 | 22.0<br>21 | 23.0<br>22 | 24.0<br>23 | 25.0<br>24 | | | | | |

Read left to right; wrapped lines continue the same array.

Element offsets: 1, 6, 11, 16, 21

Byte offsets: 4, 24, 44, 64, 84

**Stride 5 elements = 20 bytes**



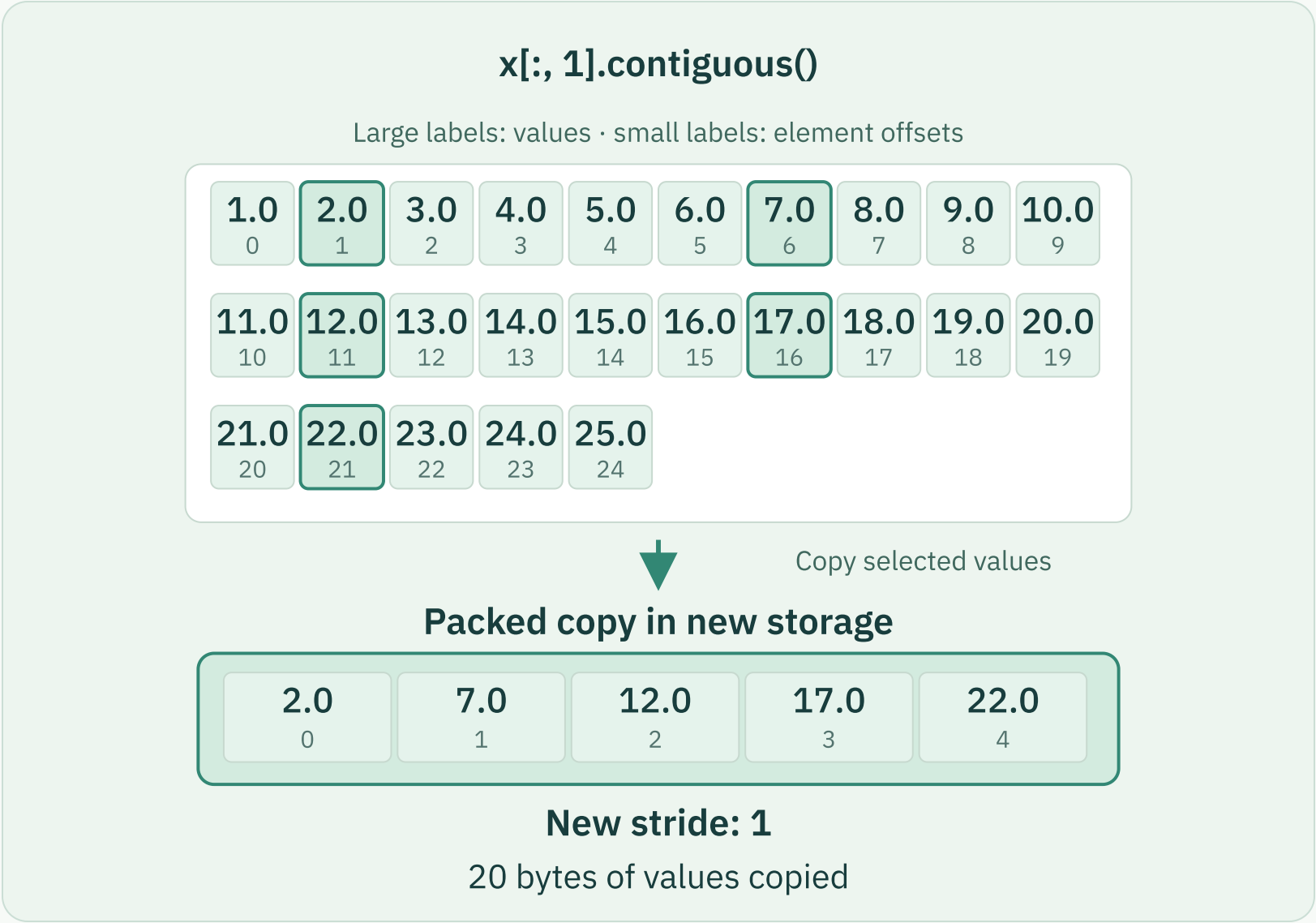
Copyright © 2026 Christopher Snow

5 / 8

# Make a packed copy.

Same selected values. New consecutive positions.

> This column needs new storage to become contiguous.






Copyright © 2026 Christopher Snow

6 / 8

# A contiguous view can share storage.

Consecutive values do not always need another copy.

> Contiguous describes value order, not storage ownership.

## x[1, :].contiguous()

Large labels: values · small labels: element offsets




Read left to right; wrapped lines continue the same array.

### Stride 1 · same x storage

contiguous() can return this row without copying.

0 bytes of values copied



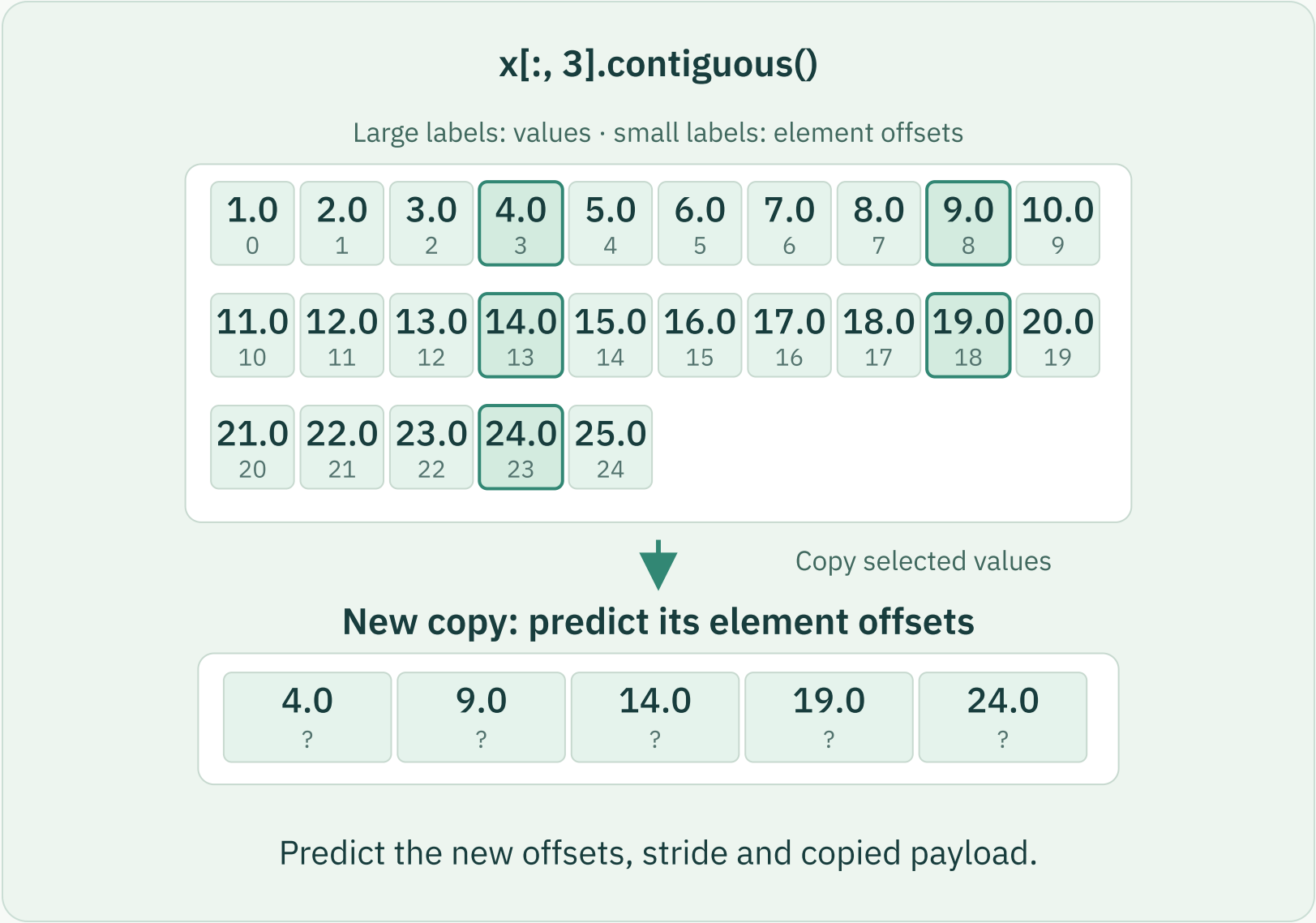
Copyright © 2026 Christopher Snow

7 / 8

# Pause: pack a different column.

Column three. Same four-byte value format.

> Pause before checking the new copy's positions.




Predict the new offsets, stride and copied payload.



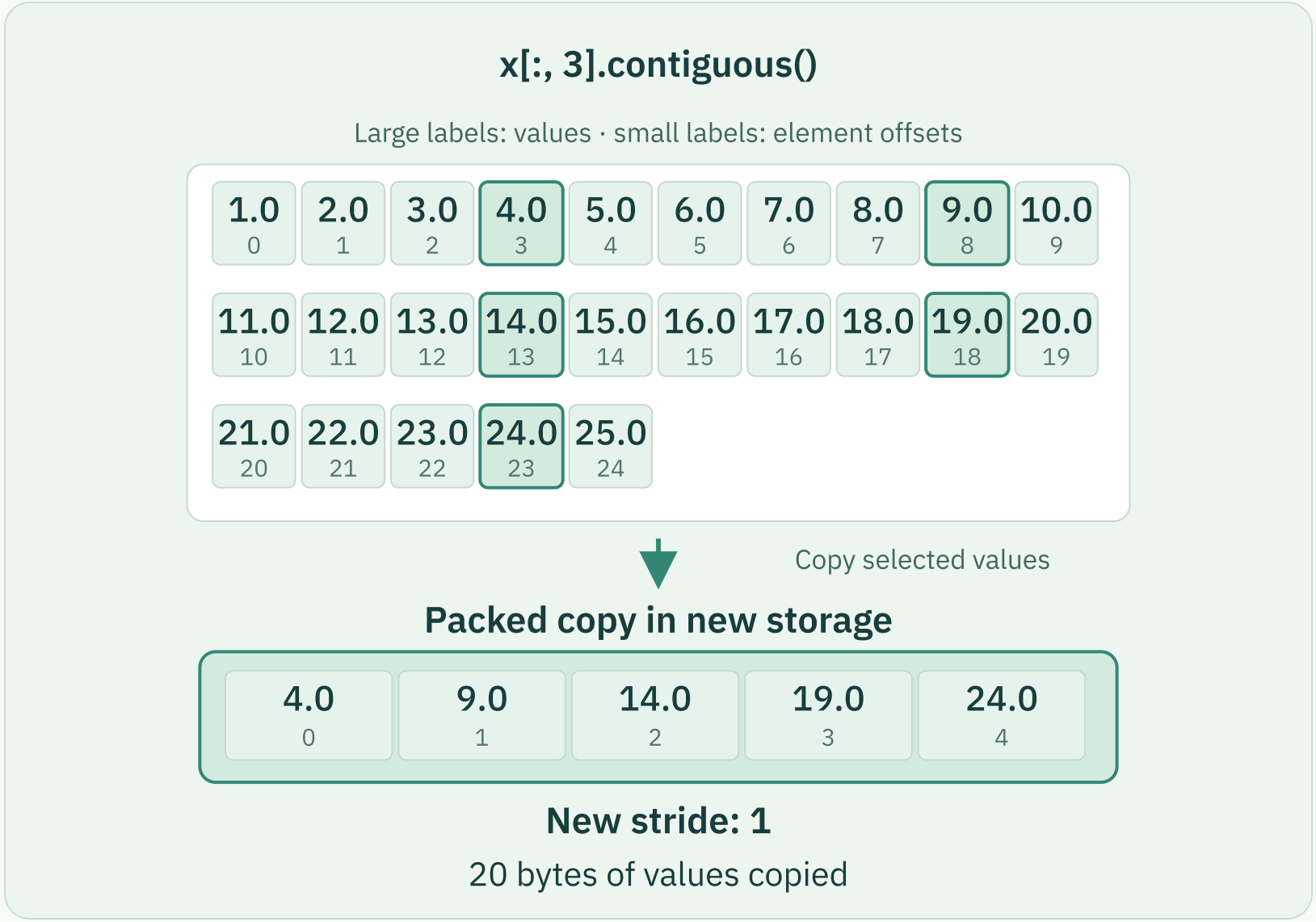
Copyright © 2026 Christopher Snow

8 / 8

# New offsets belong to new storage.

The original tensor keeps its values and positions.

> Return to Chapter 5 and explain the view/copy distinction.






Copyright © 2026 Christopher Snow